Introduzione al Reinforcement Learning

a.a. 2024/2025

Mario Sassano

Dipartimento di Ingegneria Civile e Ingegneria Informatica
Università di Roma Tor Vergata

Nelle lezioni precedenti...

- Abbiamo studiato l'equazione di Bellman per determinare la Funzione Valore

$$\begin{cases} V_k(x_k) & = \min\limits_{u_k \in U_k} \{g_k(x_k, u_k) + V_{k+1}(\underbrace{f(x_k, u_k)}_{x_{k+1}})\}, \forall x_k \in X, k = \{0, 1, ..., N-1\} \\ V_N(x_N) & = g_N(x_N), \forall x_N \in X \end{cases}$$

- Abbiamo detto che la scelta ottima ad ogni istante k è quella *greedy* rispetto alla Funzione Valore

$$u_k \in \arg\min_{u \in U_k} \{g_k(x_k, u) + V_{k+1}(f(x_k, u))\}$$

- Abbiamo visto che per risolvere l'equazione della Programmazione Dinamica è necessaria una conoscenza esatta del modello e della funzione di costo/ricavo

Reinforcement Learning (RL)

Problemi di apprendimento in cui un **agente** impara come mappare ciascuno **stato** dell'ambiente in un'**azione** corrispondente (ovvero, *cosa fare*) per massimizzare la **somma delle ricompense** ricevute

Idea di RL

All'agente non viene detto esplicitamente quale azione eseguire, ma deve imparare dall'esperienza quale azione fornisce la ricompensa maggiore **provando ad eseguirle**

- RL è diverso da *Apprendimento Supervisionato*, in cui l'agente impara da una serie di "esempi" di azioni corrette suggerite da un utente esperto
- RL è diverso da *Apprendimento non-Supervisionato*, in cui si vuole determinare pattern "nascosti" in un insieme di dati

Reinforcement Learning (RL)

Problema di apprendimento in cui un **agente** impara come mappare ciascuno **stato** dell'ambiente in un'**azione** corrispondente (ovvero, *cosa fare*) per massimizzare la **somma delle ricompense** ricevute

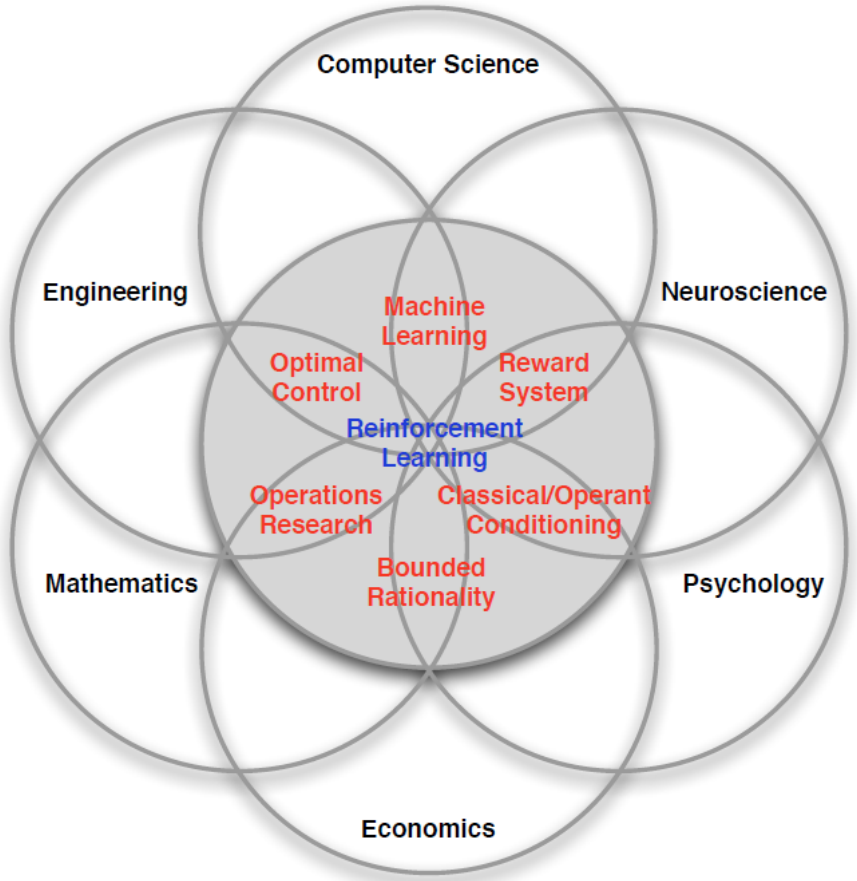
RL terzo *paradigma* di Machine Learning, caratterizzato da:

- Goal-oriented learning
- Assenza di *supervisor* (solo ricompense)
- Importanza del *tempo* (la ricompensa potrebbe essere ritardata...)
- Trade-off tra *exploration*[1] e *exploitation*[2]

[1] esplorare possibili azioni ammissibili per imparare nuove possibili scelte

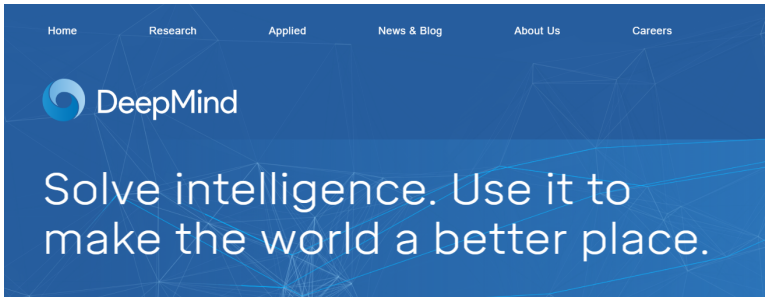
[2] sfruttare quello che si è imparato fino a quel momento per massimizzare la ricompensa ottenuta

Dalle slides del prof. D. Silver, UCL



⇒ come funziona il processo di prendere una decisione?
⇒ quale è il modo di scegliere il comportamento migliore?

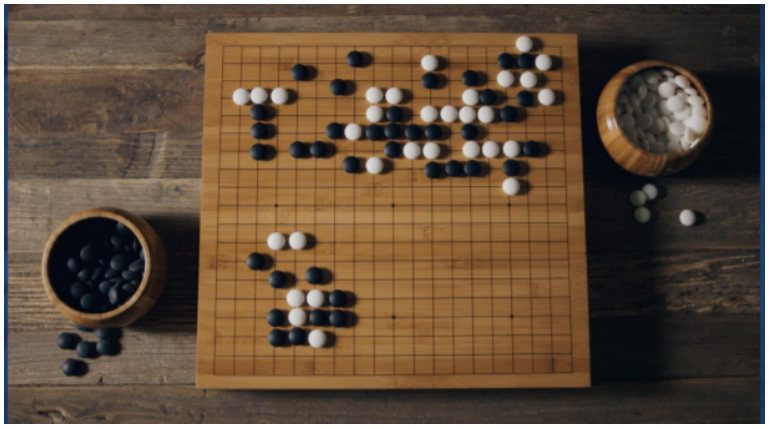
Esempi di RL e approfondimento



- Comprata da Google per oltre 500 milioni di dollari nel 2014
- Sviluppo algoritmi di Intelligenza Artificiale per Google

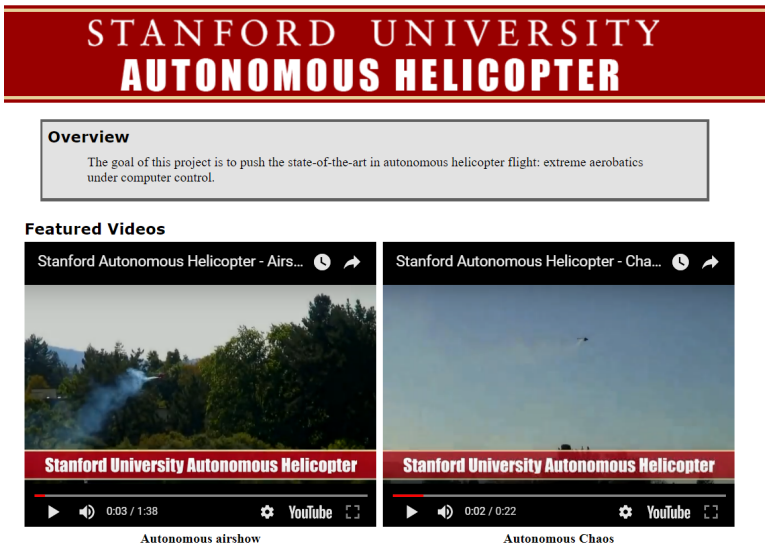
  $\Rightarrow$ suggerimenti personalizzati di ricerca
- Sviluppo di **AlphaGo** con Deep Reinforcement Learning (doc su Netflix)

  $\Rightarrow$ sconfitto il campione del mondo di **Go** (gioco con 10^{170} stati) nel 2016



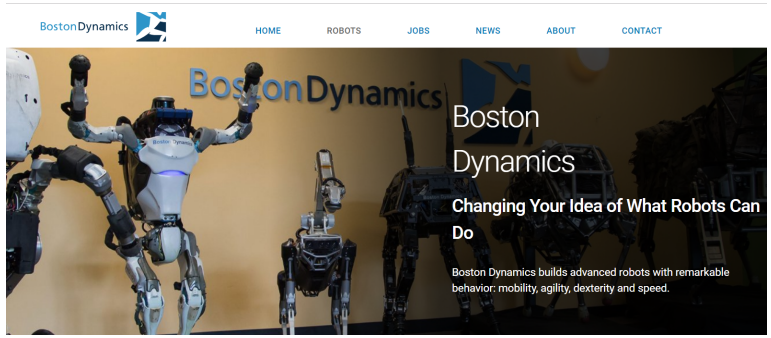


Esempi di RL e approfondimento



Guida autonoma di elicottero

Robot antropomorfi



Approfondimento nel corso del prof. D. Silver (UCL e Deepmind)

Lezioni disponibili su YouTube (RL course by David Silver)

Esempi di RL in Economia

ARTIFICIAL INTELLIGENCE, ALGORITHMIC PRICING AND COLLUSION†

EMILIO CALVANO*‡, GIACOMO CALZOLARI&‡§, VINCENZO DENICOLÒ*§, SERGIO PASTORELLO*

DECEMBER 2018

Pricing algorithms are increasingly replacing human decision making in real marketplaces. To inform the competition policy debate on possible consequences, we run experiments with pricing algorithms powered by Artificial Intelligence in controlled environments (computer simulations). In particular, we study the interaction among a number of Q-learning algorithms in the context of a workhorse oligopoly model of price competition with Logit demand and constant marginal costs. We show that the algorithms consistently learn to charge supra-competitive prices, without communicating with each other. The high prices are sustained by classical collusive strategies with a finite punishment phase followed by a gradual return to cooperation. This finding is robust to asymmetries in cost or demand and to changes in the number of players.

Keywords: Artificial Intelligence, Pricing-Algorithms, Collusion, Reinforcement Learning, Q-Learning.

J.E.L. codes: L41, L13, D43, D83.

⇒ Dynamic Pricing in oligopolio tramite RL impara a colludere tacitamente

"Dizionario" Informatica-Teoria dei Sistemi

Differente notazione per lo stesso problema ha comportato sviluppi (quasi) indipendenti:

Agente → Sistema di controllo

Azione → controllo

Ambiente → Sistema

Funzione valore ottima → funzione valore

Learning → risolvere un problema di DP senza modello

Policy → legge di controllo in retro-azione

Ricavo complessivo → (opposto di) funzionale di costo

Ricompensa → (opposto di) costo corrente

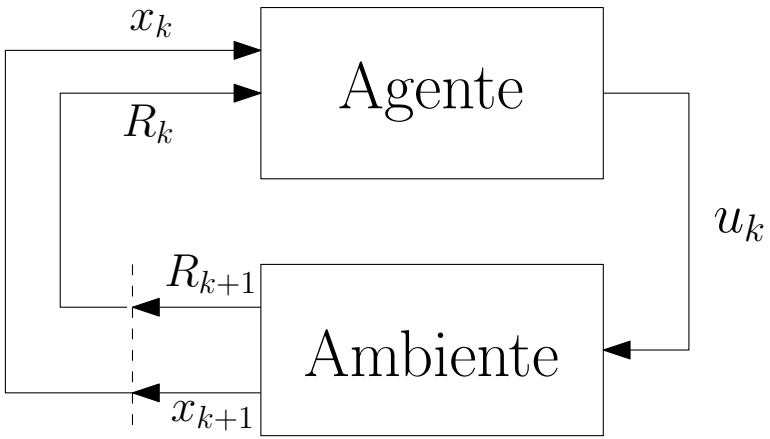
Notazione diversa può causare problemi...

$$\dot{x} = Ax + Bu, \qquad y = Cx \qquad \text{Vs} \qquad \dot{x} = Fx + Gu, \qquad y = Hx$$

Elementi principali di Reinforcement Learning

- **Policy**: definisce il comportamento dell'agente per ogni istante di tempo e in funzione dello stato corrente
- **Ricompensa**: (reward signal) definisce l'obiettivo di RL; ad ogni istante l'ambiente invia all'agente un segnale di ricompensa, in funzione di stato corrente e dell'azione eseguita (anche in maniera indiretta attraverso la modifica dello stato)
- **Funzione valore**: specifica quali sono le azioni migliori nel lungo periodo (quantità totale di ricompensa che possiamo ottenere a partire da un determinato stato). Uno stato potrebbe fornire una ricompensa bassa (breve periodo) ma avere un elevato valore (lungo periodo)
- **Modello**: (dell'ambiente) da utilizzare eventualmente per pianificare scelte future, ma siamo interessati a metodi model-free..

Schema di Reinforcement Learning



Agente si trova in

$x_k \in X$ (insieme dei possibili stati)

e sceglie l'**azione**

$u_k \in U$ (insieme delle possibili azioni)

e (in parte) come conseguenza della sua azione riceve una **ricompensa**

$R_{k+1} \in \mathbb{R}$

Markov Decision Process (f, π, ϱ) - caso deterministico

- Il comportamento dell'ambiente è descritto da un *modello* (azione → nuovo stato)

$$f : X \times U \to X, \qquad x_{k+1} = f(x_k, u_k)$$

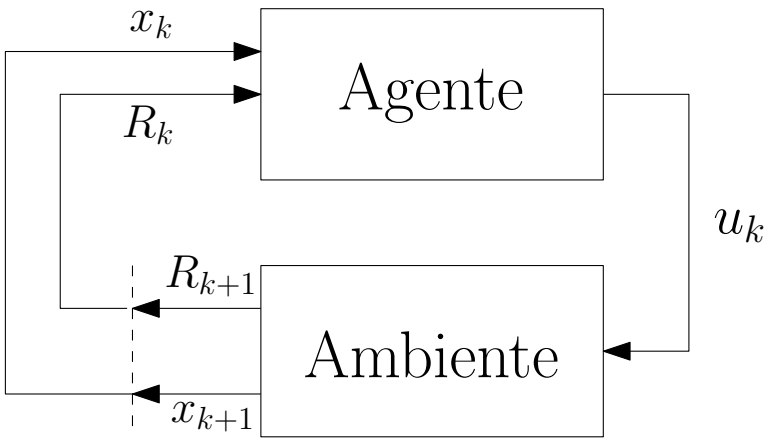
- Il comportamento dell'agente è descritto da un *policy* (stato → azione)

$$\pi : X \to U, \qquad u_k = \pi(x_k)$$

- L'ambiente genera una ricompensa/*reward* (stato, azione → ricompensa)

$$\varrho : X \times U \to \mathbb{R}, \qquad R_{k+1} = \varrho(x_k, u_k)$$

Schema di Reinforcement Learning



Agente si trova in

$x_k \in X$ (insieme dei possibili stati)

e sceglie l'**azione**

$u_k \in U$ (insieme delle possibili azioni)

e (in parte) come conseguenza della sua azione riceve una **ricompensa**

$R_{k+1} \in \mathbb{R}$

Markov Decision Process (f, π, ϱ)

- Il comportamento dell'ambiente è descritto da un *modello* (azione → nuovo stato)

$$f : X \times U \to X, \qquad x_{k+1} = f(x_k, u_k)$$

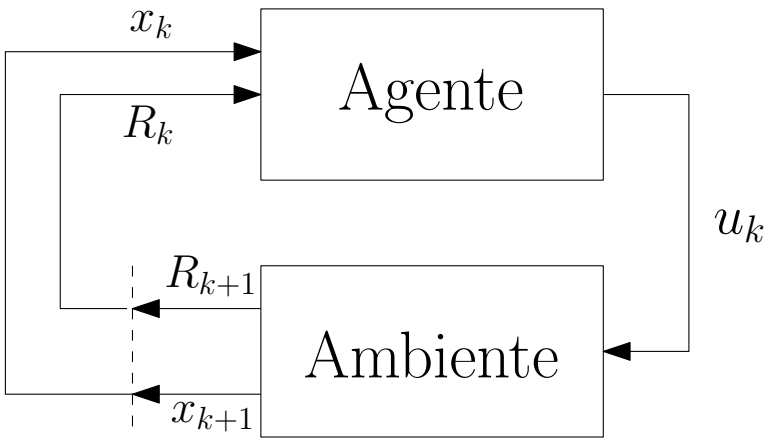
Proprietà Markoviana dello stato

In RL, lo stato deve essere tale da riassumere in maniera compatta tutta l'informazione passata in modo da determinare in maniera univoca il valore successo

Il futuro non dipende dal passato, dato il valore presente

⇒ proprietà tipica dello stato in Teoria dei Sistemi

Schema di Reinforcement Learning



Agente si trova in

$x_k \in X$ (insieme dei possibili stati)

e sceglie l'**azione**

$u_k \in U$ (insieme delle possibili azioni)

e (in parte) come conseguenza della sua azione riceve una **ricompensa**

$R_{k+1} \in \mathbb{R}$

Markov Decision Process (f, π, ϱ)

- Il comportamento dell'ambiente è descritto da un *modello* (azione → nuovo stato)

$$f : X \times U \to X, \qquad x_{k+1} = f(x_k, u_k)$$

- Il comportamento dell'agente è descritto da un *policy* (stato → azione)

$$\pi : X \to U, \qquad u_k = \pi(x_k)$$

Policy stazionaria

Per ciascuno stato fornisce sempre la stessa azione indipendentemente dal punto della sequenza in cui si incontra lo stato

Obiettivo di Reinforcement Learning

Obiettivo: massimizzare la somma delle ricompense ricevute, ovvero il **ricavo** complessivo

$$G_k(x) = R_{k+1} + R_{k+2} + \ldots + R_N \tag{1}$$

dall'istante[3] k e dallo stato $x \in X$

La definizione (1) di $G_k(x)$ ha senso solo se esiste una conclusione *ragionevole* del problema (N finito)

$\Rightarrow$ apprendimento **episodico** (scacchi, dama, ...): quando viene raggiunto uno *stato terminale* in X^+, lo stato viene *resettato* ad uno stato iniziale in X

Nel caso di apprendimento **continuo** ($N = \infty$)

$$G_k(x) = R_{k+1} + \gamma R_{k+2} + \gamma^2 R_{k+3} + \ldots. = \sum_{i=0}^{\infty} \gamma^i R_{k+i+1} \tag{2}$$

$0 \leqslant \gamma \leqslant 1$, fattore di sconto

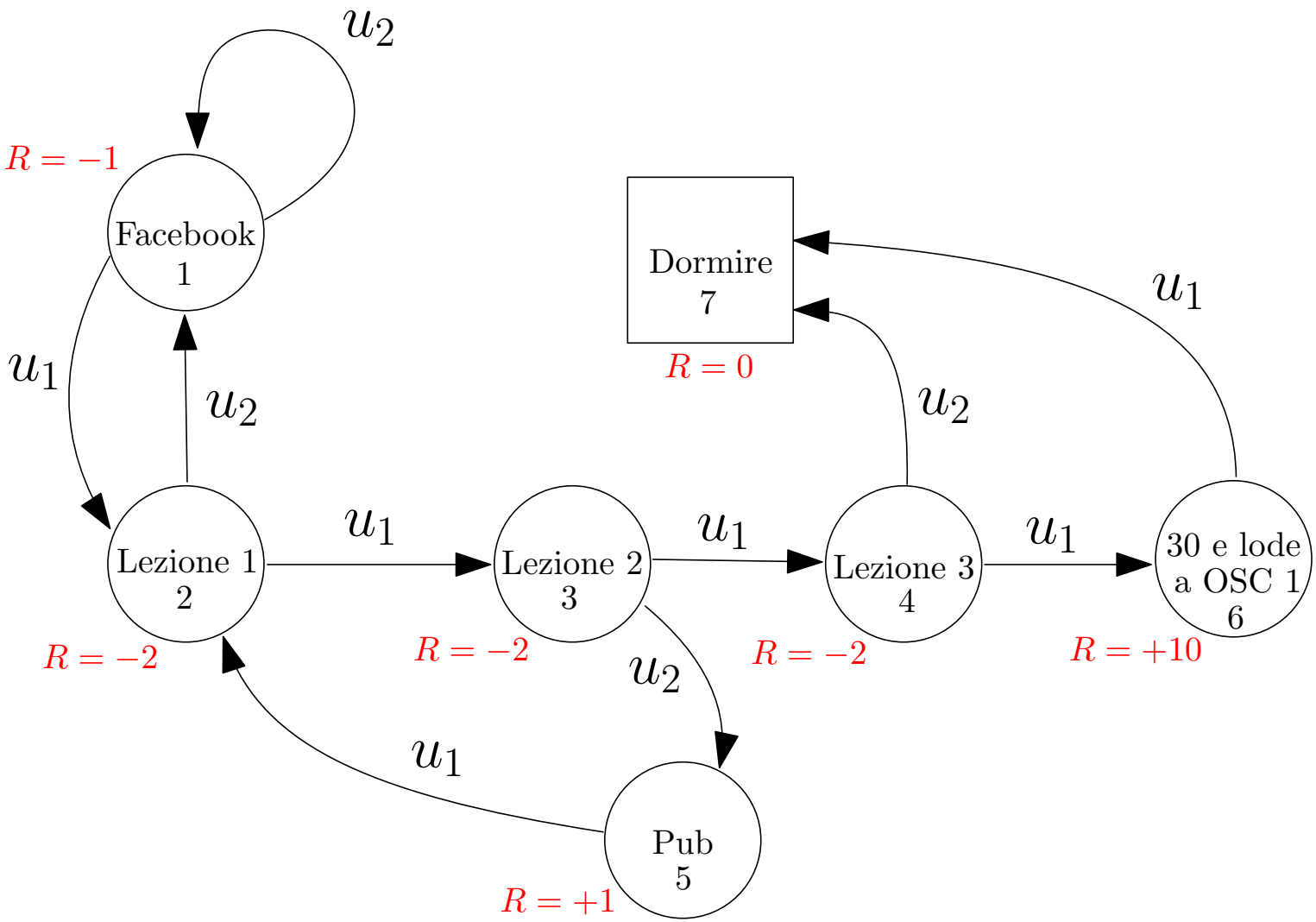
$\Rightarrow$ se $\gamma = 0$ l'agente è interessato solo alla ricompensa immediata (*strategia greedy*)...

[3] Gli istanti $k = 0, 1, 2, \ldots$ non sono necessariamente *temporali*, ma possono ad esempio rappresentare anche *turni* in un gioco

Esempio: il dilemma dello studente

Esempio adattato (stocastico → deterministico) dalle lezioni del prof. Silver

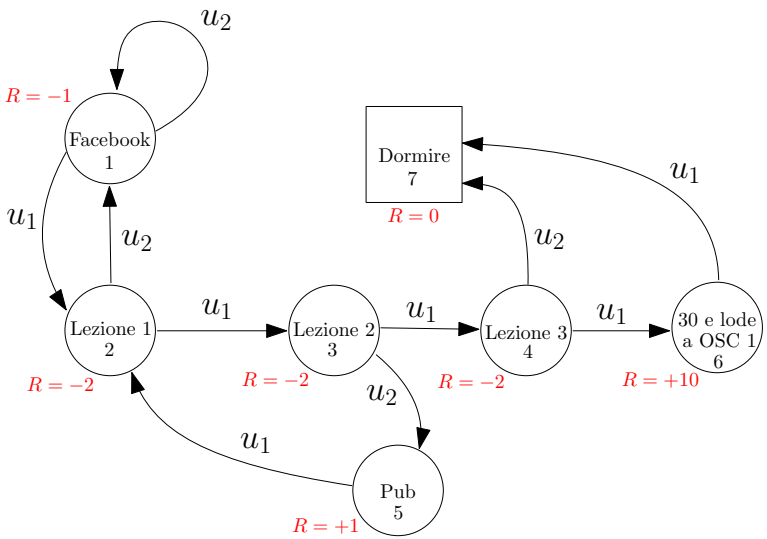
2 possibili azioni: *studiare* (u_1) o *divertirsi* (u_2)



Esempio: il dilemma dello studente

Esempio adattato (stocastico → deterministico) dalle lezioni del prof. Silver

2 possibili azioni: *studiare* (u_1) o *divertirsi* (u_2)



◯: stati in X

□: stati terminali X^+

Possibili Episodi

L1-Fb-Fb-Fb- ... ($\gamma < 1$ necessario..)

L1-L2-Pub-L1-L2-L3-30elode-Dormire

⋮

Nelle prossime lezioni...

Siamo interessati a studiare tecniche di Reinforcement Learning

Prossimi passi:

- forniamo una caratterizzazione del *valore* di ciascuno stato e/o azione data una policy
- studiamo strategie che permettano di migliorare iterativamente il valore di uno stato per una policy fissata
- eseguiamo i passi precedenti in assenza di un modello, ovvero senza conoscere f e ϱ ma solo il valore R_{k+1}